Login page references

User

I need login page references for this project

Assistant

Curated set of login page references, tailored to the stack (React + TS + MUI + react-hook-form).

Official MUI templates — most directly useful since they match the component library: - MUI Sign-in template (official, light/dark mode, source downloadable via CodeSandbox/StackBlitz)

MUI + React login examples on GitHub: - Login-Page-MUI — clean form with remember-me, password recovery, sign-up link - react-material-login-ui-template — Devias/Material UI sign-in demo - material-kit-react — broader kit with a login page included

Design inspiration galleries (layout/UX patterns, not code): - 99designs login inspiration - Muzli — 60+ login screen examples - Justinmind — 40+ login page examples

2026 pattern notes for a movies app: dual-pane layouts (form on one side, branding/value prop on the other), the "or continue with email" divider convention if social login is added, minimal fields with clear error states via `react-hook-form`. Noted that `references.txt` already has a localStorage-auth tutorial saved, and there's an empty `LoginForm` component folder waiting to be built.

User

[MUI Sign-in template] — how do I download it ?

Assistant

Two sign-in options — plain "Sign-in" and "Sign-in side" (two-column layout), each with a live preview page and GitHub source folder. On the live preview page, a top-right toolbar has a GitHub icon and CodeSandbox/StackBlitz icons — GitHub goes to the source folder (browse files individually, or "Code → Download ZIP" for the whole material-ui repo then copy the one template folder), or open directly in CodeSandbox/StackBlitz to explore and export without cloning. Given the project's "don't code for me" rule, this is meant as a reference to look at and adapt rather than something to wire in directly.

Assistant

Confirmed: the project is on MUI v5.15.6, but the sign-in-side template code was generated for MUI v6/v7 docs, where `PaletteMode` is re-exported from `styles/index.d.ts`. In the installed v5, `PaletteMode` only lives at the package root (`@mui/material`), not under `@mui/material/styles`. Two paths, since coding isn't done directly: change the import in `themePrimitives.ts` to pull `PaletteMode` from `@mui/material` instead (smaller fix), or upgrade `@mui/material`/`@mui/icons-material`/`@emotion/*` to v6+ so the template's imports match as-written (bigger change, may ripple into other v5-specific APIs). Given this is one line in one file, adjusting the import is the smaller fix — worth checking `themePrimitives.ts`/`AppTheme.tsx` for other v6-only APIs like `colorSchemes` in `createTheme`.

User

how to upgrade

Assistant

Upgrade path, since v6 was designed as low-breakage from v5: (1) update core packages — `npm install @mui/material@6 @mui/icons-material@6 @mui/system@6 @mui/utils@6`; (2) check React/TS minimums (v6 needs React 17, TS 4.7 — project's on React 18.2, fine; also pin `react-is` via `overrides`

if on React 18 or below); (3) run codemods for grid-v2-props, list-item-button-prop, theme-v6; (4) manually check things codemods don't cover — `Typography`'s `color` prop no longer a system prop, `theme.applyStyles('dark', {...})` replacing manual mode checks; (5) rebuild and let `tsc` surface anything left.

Worth deciding first: MUI is actually at v9 now, with v7 and v9 migration guides layered on top of v6 — going exactly to v6 might not be enough if the sign-in-side template leans on v7 conventions. Given this is an assignment, worth checking how much of the rest of the app depends on v5-only APIs before committing to a full jump — maybe on a branch first.

User

I need to check which packages have updates , in general

Assistant

Ran `npm outdated`. MUI-related: `@mui/material`/`@mui/icons-material` at 5.15.6 → wanted 5.18.0 → latest 9.3.1 (small bump within range, major jump needs manual `package.json` edit); `@emotion/*` minor patches available. React core: `react`/`react-dom` 18.2.0 → wanted 18.3.1 → latest 19.2.8 (major jump needs its own compatibility check); `react-hook-form` 7.49.3 → 7.85.0 safe within range; `react-router-dom` 6.21.3 → wanted 6.30.4 → latest is v7 (major). Tooling: Storybook, ESLint, TypeScript, Vite all have newer majors available, separate migration efforts. `npm update` picks up everything safe within current ranges; anything beyond "Wanted" needs manual `package.json` edits plus migration guides.

User

[pasted npm outdated table, then] could you check my code please ?

Assistant

Ran `npx tsc --noEmit` — 107 TypeScript errors in five buckets:

1. MUI version mismatch (~90 errors) — `src/shared-theme/*`, `signIn.tsx`, `signUp.tsx`, `forgotPassword/index.tsx`: `Property 'vars'/'applyStyles' does not exist on type 'Theme'`, `'PaletteMode' has no exported member`, `'cssVariables' does not exist in type 'ThemeOptions'`. Same root cause as before — clears once MUI is upgraded.
2. `forgotPassword/index.tsx` — two extra issues: `paper` isn't a valid slot prop on `Dialog`'s `slotProps` in the installed version; `React.SubmitEvent` doesn't exist, likely meant `React.FormEvent` or `React.SyntheticEvent`.
3. `src/api/tmdb-api.ts` — `import.meta.env` not typed (6 errors), unrelated to login work, pre-existing on the branch (confirmed via diff); no `types: ["vite/client"]` in `tsconfig.json` and no `vite-env.d.ts` in `src/`.
4. Unused variables/imports (TS6133/TS6196), 7 spots across `movieCard/index.tsx`, `index.tsx`, `addMovieReviewPage.tsx`, `homePage.tsx`, `movieCard.stories.tsx`, `movieListHeader.stories.tsx`, `movieList.stories.tsx` — `noUnusedLocals`/`noUnusedParameters` are on.
5. `util.ts:1` — missing types for `lodash/truncate`, needs `@types/lodash`.

Separate from TS errors: `git diff --stat` shows ~48 files with near-total line rewrites but identical content — CRLF line-ending churn (confirmed `src/util.ts` is CRLF vs repo's original LF). Worth checking `core.autocrlf` or adding `.gitattributes`, otherwise every PR/diff review on this branch will be unreadable noise.

User

how can I update one by one , to see the errors ?

Assistant

Best approach: one package (or tightly-coupled pair) per commit, checking `tsc` after each. Setup: commit a clean baseline first (`git add -A && git commit -m "wip before dependency upgrades"`). Order — safest first: (1) `@types/lodash` (zero risk); (2) `react-hook-form` → wanted version (low risk); (3) `react-router-dom` → wanted 6.30.4, stay on v6 for now; (4) `@emotion/react`/`@emotion/styled` → wanted versions; (5) `@mui/material` + `@mui/icons-material` together (must stay in lockstep) — clears/changes most of the 90 theme errors; (6) dev tooling last (`typescript`, `eslint`, `@typescript-eslint/*`, `storybook`, `vite`). Per-package loop: `npm install <package>@<version>`, `npx tsc --noEmit`, compare error count, commit if clean or roll back if messy. For MUI itself (major jump), explicit version install then codemods then recheck.

User

if I install the latest MUI. what do you think ?

Assistant

Good news: `@mui/material@9.3.1` still peer-supports React 18, so no React 19 jump needed just for latest MUI. Actual take: going straight to latest is probably the better call here, not the riskier one — the sign-in-side template was pulled live from mui.com's docs, versioned against v9.3.1, and the `vars`/`applyStyles`/`cssVariables` errors are v6+ APIs that persist essentially unchanged through v7 and v9. Stopping at v6 would just trade one mismatch for a smaller different one. Where "one version at a time" pays off is bumping several unrelated packages with independent breaking-change surfaces — here it's really one upgrade (MUI + its CSS-vars theming model). Practical caveats: do it on the `feature/loginPage` branch with a clean commit first; `@mui/icons-material`/`@mui/system`/`@mui/utils` must move in lockstep with `@mui/material`; codemods are pinned per version so going straight to latest means more manual cleanup on the ~10 non-theme errors; add the `react-is` override for React 18 compatibility.

User

what do you recommend? I would go for the latest for everything. not sure if worth it

Assistant

Confirmed ESLint v9→v10 completely removed the old `.eslintrc.*` format (project uses `.eslintrc.cjs`), so jumping to ESLint 10 means a full flat-config rewrite plus matching major bumps for `eslint-plugin-storybook`, `@typescript-eslint/*`, `eslint-plugin-react-hooks`.

Honest recommendation: don't go latest-for-everything, scope it to what the login page actually needs.

Do now — upgrade to latest: `@mui/material` + `@mui/icons-material` + `@mui/system` + `@mui/utils` → v9 (needed, template written against it, React 18 still supported); `@types/lodash` (unrelated small fix); `react-hook-form`, `@emotion/react`/`styled` → in-range "wanted" versions (trivial).

Defer, separate from this branch: React 19 (not required, has its own breaking changes touching files across the whole app); react-router-dom v7 (different routing paradigm, unrelated to login page); ESLint 8→10 (config rewrite project, zero benefit to login page); Storybook 8→10, Vite 5→8, TypeScript 5→7 (tooling majors, no functional payoff for this task).

Reasoning: on an assignment deadline, every deferred upgrade is a self-contained migration project with its own blast radius unrelated to getting the login page working — bundling them means debugging six

unrelated breaking-change sets at once instead of one. Do the MUI bump now, get the login page compiling and merged, treat tooling upgrades as separate follow-up work.

User

[pasted the "Do now" list again, trailing empty bullet]

Assistant

Looks like the message might've gotten cut off — trailing empty bullet. Asked whether to go ahead and run the installs now, or whether something was about to be added to the list.

User

guide me on this [pasted the "Do now" list again]

Assistant

Sequence, in order, commit first to diff/revert against:

```
git add -A && git commit -m "wip before MUI v9 upgrade"
```

1. Install the MUI v9 set together (must move in lockstep): `npm install @mui/material@9 @mui/icons-material@9 @mui/system@9 @mui/utils@9`
2. Pick up the trivial ones: `npm install --save-dev @types/lodash` and `npm install react-hook-form@latest @emotion/react@latest @emotion/styled@latest`
3. Add the React 18 compatibility override (MUI v9 pulls in `react-is@19` internally): `npm install react-is@18.3.1`, then add `"overrides": { "react-is": "^18.3.1" }` to `package.json`
4. Check for deprecated-prop usage before running codemods — MUI v9 removed a long list of deprecated props (`PaperProps`, `componentsProps`, `TransitionComponent`, `BackdropComponent`, `ChipProps`, `imgProps`, etc.) in favor of `slots`/`slotProps`, no single "run everything" codemod, so grep first: `grep -rn "PaperProps\|componentsProps\|TransitionComponent\|BackdropComponent\|ListboxComponent\|PopperC src/` — if matches found, run only the matching codemod(s)
5. Check for legacy Grid usage (removed in v9): `grep -rn "from '@mui/material/Grid'\|GridLegacy" src/`, run `npx @mui/codemod@latest v6.0.0/grid-v2-props src` if found
6. Re-check: `npx tsc --noEmit`, compare against the 107-error baseline
7. Commit once clean: `git add -A && git commit -m "upgrade MUI to v9, react-hook-form/emotion to latest"`

Offered to run the grep checks in steps 4–5 immediately so codemod needs would be known upfront before installing.

User

[pasted grep result: only one match, `PaperPropsVariantOverrides` in themePrimitives.ts]

Assistant

That's a false positive — `PaperPropsVariantOverrides` is a module-augmentation interface for adding a custom `variant` value (`highlighted`) to the `Paper` component, not usage of the deprecated `PaperProps` object prop the v9 codemod targets. Nothing to fix there. The Grid-legacy check also came back clean — no `GridLegacy` or old `xs`/`sm`/`md`-style Grid imports anywhere in `src/`. So: no codemods needed for either deprecated-props or Grid — clear to go straight from step 1 (install) to step 6 (`tsc --noEmit`); steps 4 and 5 are done and both negative.